



SKILL WORKBOOK

24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

TEAM FSAD
K L UNIVERSITY | VADDESWAREM



SKILL WORKBOOK

24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L - FSAD

STUDENT NAME	Manan Shah
STUDENT ID	2400090137
YEAR	2nd
SEMESTER	EVEN
FACULTY NAME	SRIDEVI SAKHAMURI

DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L
FULL STACK APPLICATION DEVELOPMENT

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

#SKILL - 2 → Hibernate CRUD Operations

Prerequisites:

- **Basic knowledge of Java**
- **Understanding of classes, objects, and OOP**
- **Familiarity with SQL basics**

A retail inventory system needs to store product details such as **productId**, **name**, **description**, **price**, and **quantity**. The admin should be able to **add new products**, **retrieve product information**, **update price or quantity**, and **delete discontinued products**.

Your task is to **implement complete CRUD operations** using **Hibernate** with proper entity mapping using **JPA annotations**.

1. Create a **Product** entity with fields: **id (primary key)**, **name**, **description**, **price**, **quantity**.
2. Configure **Hibernate** and map the Product entity to a table using **JPA annotations**.
3. Insert **multiple Product records** into the database.
4. Retrieve a product using its **id**.
5. Update the **price** or **quantity** of any selected product.
6. Delete a product record by **id** if it is discontinued.
7. **Note:**
 - a. Implement Hibernate **ID generation strategies (AUTO, IDENTITY, SEQUENCE)** inside the entity to observe how primary key values differ.
 - b. The **ID can be manual or auto generated**, based on your chosen strategy.
8. **Push the Hibernate project into a single GitHub repository.**

Src/main/java

Com.inventory.app

MainApp.java

```
package com.inventory.app;

import java.util.Scanner;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

import com.inventory.model.Product;
import com.inventory.util.HibernateUtil;

public class MainApp {

    static SessionFactory factory = HibernateUtil.getSessionFactory();
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {

        int choice;

        do {
            System.out.println("\n===== PRODUCT MENU =====");
            System.out.println("1. Add Product");
            System.out.println("2. View Product");
            System.out.println("3. Update Product");
            System.out.println("4. Delete Product");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");

            choice = sc.nextInt();
```

```

switch (choice) {
    case 1:
        insertProduct();
        break;
    case 2:
        viewProduct();
        break;
    case 3:
        updateProduct();
        break;
    case 4:
        deleteProduct();
        break;
    case 5:
        System.out.println("Thank you!");
        break;
    default:
        System.out.println("Invalid choice");
}

} while (choice != 5);
}

// ===== INSERT =====
static void insertProduct() {

    Session session = factory.openSession();
    Transaction tx = session.beginTransaction();

    Product p = new Product();

    sc.nextLine(); // clear buffer
    System.out.print("Enter Product Name: ");
    p.setName(sc.nextLine());

    System.out.print("Enter Description: ");
    p.setDescription(sc.nextLine());

    System.out.print("Enter Price: ");
    p.setPrice(sc.nextDouble());

    System.out.print("Enter Quantity: ");
    p.setQuantity(sc.nextInt());

```

```

    session.save(p);
    tx.commit();
    session.close();

    System.out.println("Product inserted successfully");
}

// ===== VIEW =====
static void viewProduct() {

    Session session = factory.openSession();

    System.out.print("Enter Product ID: ");
    int id = sc.nextInt();

    Product p = session.get(Product.class, id);

    if (p != null) {
        System.out.println("\nProduct Details:");
        System.out.println("ID: " + p.getId());
        System.out.println("Name: " + p.getName());
        System.out.println("Description: " + p.getDescription());
        System.out.println("Price: " + p.getPrice());
        System.out.println("Quantity: " + p.getQuantity());
    } else {
        System.out.println("Product not found");
    }

    session.close();
}

// ===== UPDATE =====
static void updateProduct() {

    Session session = factory.openSession();
    Transaction tx = session.beginTransaction();

    System.out.print("Enter Product ID to update: ");
    int id = sc.nextInt();

    Product p = session.get(Product.class, id);

```

```

    if (p != null) {
        System.out.print("Enter New Price: ");
        p.setPrice(sc.nextDouble());

        System.out.print("Enter New Quantity: ");
        p.setQuantity(sc.nextInt());

        session.update(p);
        tx.commit();
        System.out.println("Product updated successfully");
    } else {
        System.out.println("Product not found");
    }

    session.close();
}

// ===== DELETE =====
static void deleteProduct() {

    Session session = factory.openSession();
    Transaction tx = session.beginTransaction();

    System.out.print("Enter Product ID to delete: ");
    int id = sc.nextInt();

    Product p = session.get(Product.class, id);

    if (p != null) {
        session.delete(p);
        tx.commit();
        System.out.println("Product deleted successfully");
    } else {
        System.out.println("Product not found");
    }

    session.close();
}
}

```

Com.inventory.model

Product.java

package com.inventory.model;

import javax.persistence.*;

@Entity

@Table(name = "product")

public class Product {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private int id;

@Column(nullable = false)

private String name;

private String description;

private double price;

private int quantity;

// Getters and Setters

public int getId() {

return id;

}

public void setId(int id) {

this.id = id;

}

public String getName() {

return name;

}

public void setName(String name) {

this.name = name;


```

    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
}

```

Com.inventory.util

HibernateUtil.java

```

package com.inventory.util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static SessionFactory factory;

```

```

static {
    try {
        factory = new Configuration()
            .configure("hibernate.cfg.xml")
            .buildSessionFactory();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static SessionFactory getSessionFactory() {
    return factory;
}
}

```

Src/main/resources

Hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<hibernate-configuration>
    <session-factory>

        <!-- JDBC Connection Settings -->
        <property name="hibernate.connection.driver_class">
            com.mysql.cj.jdbc.Driver
        </property>

        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/inventorydb1
        </property>

        <property name="hibernate.connection.username">
            root
        </property>

        <property name="hibernate.connection.password">
            Ashu@2005

```

```
</property>
```

```
<!-- Hibernate Dialect -->
```

```
<property name="hibernate.dialect">
    org.hibernate.dialect.MySQL8Dialect
</property>
```

```
<!-- Hibernate Behavior -->
```

```
<property name="hibernate.hbm2ddl.auto">
    update
</property>
```

```
<property name="hibernate.show_sql">
    true
</property>
```

```
<!-- Entity Mapping (FIXED) -->
```

```
<mapping class="com.inventory.model.Product"/>
```

```
</session-factory>
```

```
</hibernate-configuration>
```

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.klu</groupId>
    <artifactId>Experiment-2</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <dependencies>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>
    <dependency>
        <groupId>javax.persistence</groupId>
        <artifactId>javax.persistence-api</artifactId>
```

```

</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>
</dependencies>
</project>

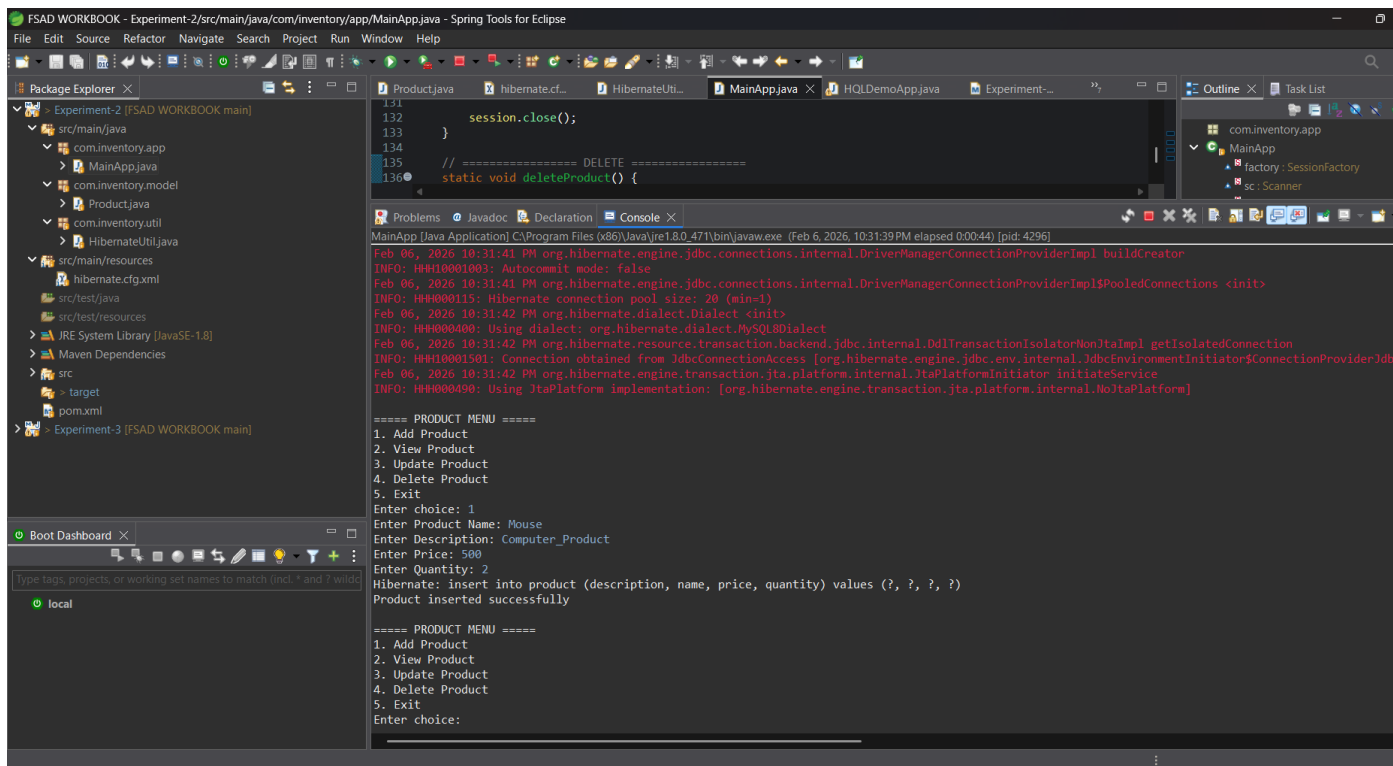
```

```

mysql> show tables;
+-----+
| Tables_in_inventorydb1 |
+-----+
| product                |
+-----+
1 row in set (0.05 sec)

mysql> |

```



VIVA QUESTIONS:

1. What is ORM and why is it used in application development?

ORM (Object Relational Mapping) is a technique that connects object-oriented programming languages like Java with relational databases. It automatically maps Java classes to database tables and objects to table records. ORM is used to reduce the need for writing complex SQL queries, make code easier to maintain, and allow developers to work with objects instead of database rows, which speeds up development.

2. Which annotation is used to mark a class as a JPA entity?

The `@Entity` annotation is used to mark a Java class as a JPA entity. This tells Hibernate or JPA that the class should be mapped to a database table and its objects should be stored as records in that table.

3. Why must every Hibernate entity have a primary key?

Every Hibernate entity must have a primary key because it uniquely identifies each record in the database table. Hibernate uses this key to retrieve, update, and delete specific records. Without a primary key, Hibernate cannot track or manage objects properly.

4. How do you update an existing record using Hibernate?

To update a record in Hibernate, you first open a session and begin a transaction. Then you retrieve the existing object from the database using its primary key, modify its values in the program, and commit the transaction. Hibernate automatically detects the changes and updates the corresponding row in the database.

5. What is the difference between `get()` and `load()` in Hibernate?

The `get()` method immediately fetches data from the database and returns the actual object; if no record is found, it returns null. In contrast, `load()` returns a proxy object without hitting the database immediately and only fetches the data when it is actually needed; if the record does not exist, it throws an exception.

(For Evaluator’s use only)

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator’s Observation</u> Marks Secured: _____ out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:
---	--

